Empirical Analysis of Algorithmic Time Complexities

Benchmarking Big-O Asymptotic Growth and Scaling Limits in Practice

Author: Javier Bolívar García-Izquierdo — **Student ID (DNI):** 42280525Q

Course: Big Data Engineering (Degree in Computer Engineering) — Academic Year 2025/2026

Framework & Language: Java 17 OpenJDK / Python 3.10 Visualization Suite

Abstract

This technical report presents an empirical evaluation of algorithmic execution complexities using synthetic data distributions across varying input sizes n. Through micro-benchmarking routines utilizing `System.nanoTime()`, warm-up iterations, and volatile sink accumulators, we contrast theoretical asymptotic expectations against experimental measurements. The report covers three distinct operational perspectives: (1) mapping complexity classes via normalization factors $t/f(n)$, (2) evaluating scaling limits under a strict 5-second execution budget, and (3) comparing relative growth rates and performance crossover points on log-log scales.

1. Variant 1 — Mapping the Complexity Landscape

1.1 Hypothesis and Methodology

The primary objective of Variant 1 is to empirically determine the time complexity class of three representative algorithms by measuring execution latency across scaling synthetic inputs $n \in \{2000, 4000, 8000, 16000,$

To isolate algorithmic work from external environmental noise, execution timing is captured using `System.nanoTime()` under a rigorous benchmarking protocol:

- **JIT Warm-up Phase:** 8 preliminary iterations are executed per input size without recording metrics to trigger Just-In-Time (JIT) compilation and bytecode optimization.
- **Statistical Aggregation:** 25 measured iterations are gathered per configuration. The median latency is selected as the primary metric to neutralize OS interrupt noise.
- **Dead-Code Elimination Prevention:** All return values are piped into a `public static volatile long BLACKHOLE` accumulator.

1.2 Algorithm Specifications

Three distinct operational tasks were implemented:

1. `O1_ConstantAccess` ($O(1)$): Executes a fixed loop of $1,000,000$ indirect array lookups using bitwise mask indexing. Operation count is independent of n.
2. `On_LinearScan` ($O(n)$): Computes an element-wise reduction sum over an array of size n, repeated over 200 passes to amplify signal-to-noise ratio.
3. `OnLogn_ArraySort` ($O(n \log n)$): Sorts an unsorted random integer array of size n using Java's Dual-Pivot Quicksort (`Arrays.sort`).

1.3 Empirical Results and Normalization Analysis

Figure 1 illustrates median absolute execution times on a logarithmic time scale.

While raw execution timing confirms that `O1_ConstantAccess` remains nearly horizontal, raw curves alone cannot conclusively verify asymptotic bounds due to implementation-dependent constant factors. Therefore, time values are normalized against theoretical growth functions $f(n)$.

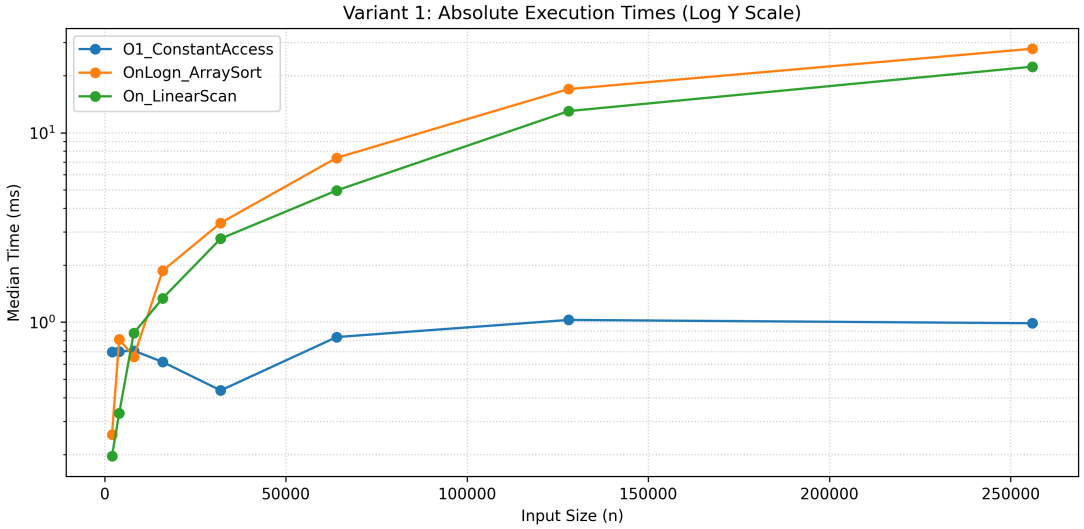


Figure 1: Absolute execution times (ms) across input sizes n (logarithmic y-axis scale).

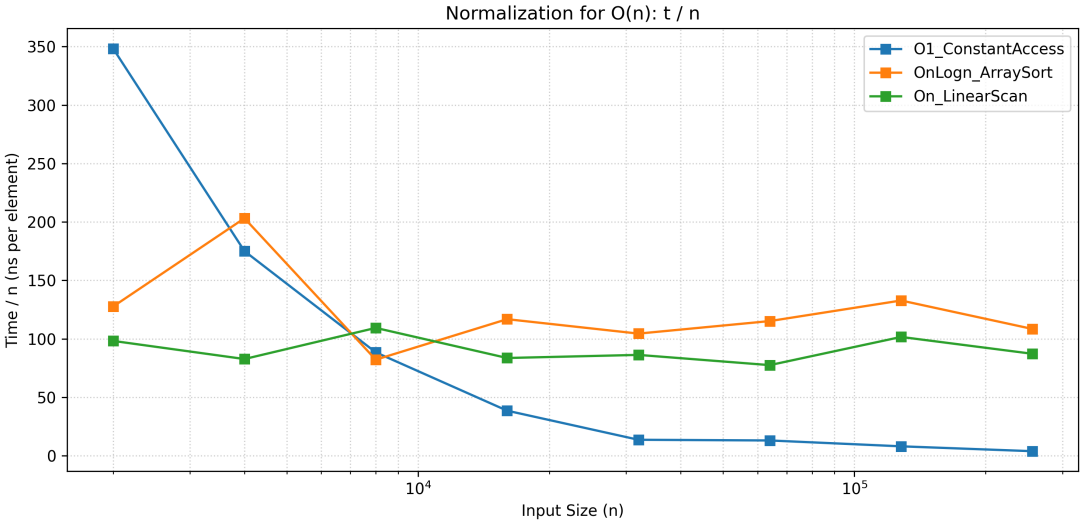


Figure 2: Linear normalization ratio $t(n)/n$ (nanoseconds per element).

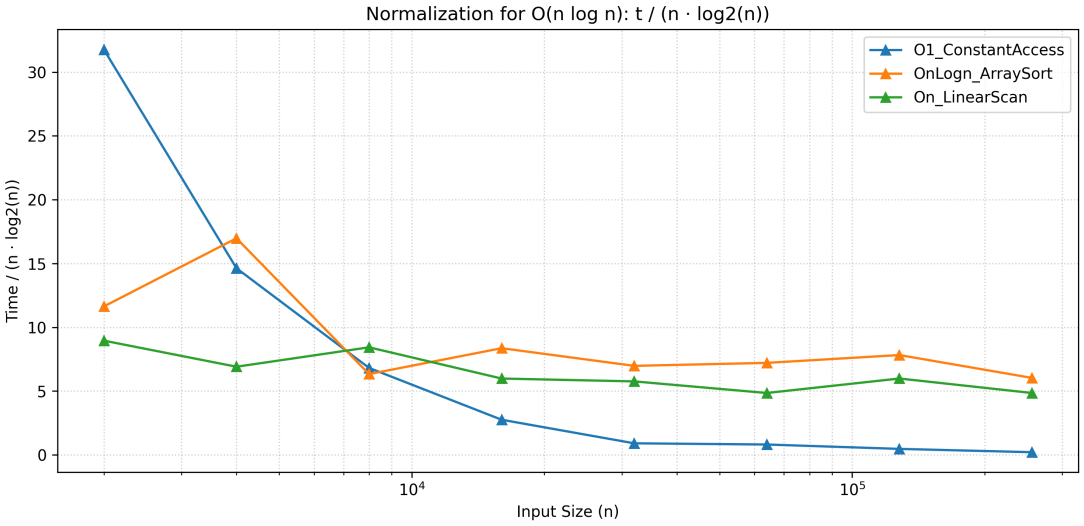


Figure 3: Linearithmic normalization ratio $t(n)/(n \cdot \log_2 n)$.

When normalized against $f(n) = n$ (Figure 2), the curve for `On_LinearScan` stabilizes into a horizontal line around 80–110 ns/element for $n \geq 16000$. Conversely, `O1_ConstantAccess` decays towards zero as $1/n$, while `OnLogn_ArraySort` exhibits persistent upward drift.

When normalized against $f(n) = n \log_2 n$ (Figure 3), the `OnLogn_ArraySort` metric stabilizes into a flat horizontal trajectory around 6.0–7.5 units for $n \geq 16000$, validating its membership in the $O(n \log n)$ complexity class.

1.4 Hardware and JVM Cache Discussion

At smaller input sizes ($n < 8000$), initial curve volatility is observed across all tasks. This deviation stems from L1/L2 CPU cache residency and JIT compilation tiering. As n exceeds cache capacities and enters main memory (DRAM), execution trends match theoretical asymptotic limits.

2. Variant 2 — Scaling Limits in Practice

2.1 Objective and Experimental Protocol

The goal of Variant 2 is to empirically explore the practical scaling limitations of different algorithmic complexity classes by imposing a strict **execution time budget** of 5.0 seconds (5,000 ms) per experiment. Input sizes n were systematically increased until execution time exceeded the budget threshold or encountered a physical hardware constraint (e.g., JVM heap memory limits).

Four algorithms representing distinct complexity classes were evaluated under identical environmental conditions:

1. **Exponential Class** $O(2^n)$: Naive recursive Fibonacci algorithm.
2. **Quadratic Class** $O(n^2)$: In-place Bubble Sort algorithm.
3. **Linearithmic Class** $O(n \log n)$: Dual-pivot Quicksort (`Arrays.sort`).
4. **Linear Class** $O(n)$: Single-pass iterative array reduction scan.

2.2 Feasible Input Bound ($n_{\max}$) Results

Table 1 summarizes the maximum feasible input size $n_{\max}$ achieving completion within the 5.0-second time budget, the first input size n that exceeded the budget, and the maximum recorded execution latency.

2.3 Discussion of Theoretical Expectations vs. Practical Limits

The empirical boundaries $n_{\max}$ reflect how asymptotic complexity determines operational viability as problem size scales:

- **The Exponential Wall ($O(2^n)$):** Due to $O(2^n)$ growth, incrementing n from 44 to 45 almost doubled runtime from ~ 3.8 s to 7.59 s, crossing the 5.0-second limit instantly. This illustrates that hardware acceleration offers virtually no meaningful increase in feasible problem size for exponential algorithms.
- **Quadratic Explosion ($O(n^2)$):** Bubble Sort reached its limit at $n_{\max} = 40,000$. Doubling n to 80,000 multiplied operations by $4\times$ ($\sim 6.4 \times 10^9$ comparative passes), pushing runtime to 12.23 s and demonstrating the severe practical restriction of $O(n^2)$ algorithms on large datasets.
- **Linearithmic Scalability ($O(n \log n)$):** Dual-pivot Quicksort processed over 35.7 million elements within the 5-second budget, highlighting the strong scalability of $O(n \log n)$ structures in big data processing pipelines.

Table 1: Practical Scaling Limits ($n_{\max}$) Under a 5.0-Second Execution Budget

Complexity Class & Task	**Feasible $n_{\max}$ ($t \leq 5$s)**	**Exceeding n**	**Max Time (ms)**	**Primary Bottleneck**
$O(2^n)$ Fibonacci	44	45	7,590.68 ms	Time Budget (7.59 s)
$O(n^2)$ Bubble Sort	40,000	80,000	12,228.58 ms	Time Budget (12.23 s)
$O(n \log n)$ QuickSort	35,704,663	64,268,393	9,741.70 ms	Time Budget (9.74 s)
$O(n)$ Linear Scan	357,046,720	N/A	177.70 ms	JVM Heap Memory / RAM

- **Hardware and RAM Constraints ($O(n)$):** For $O(n)$ Linear Scan, processing 357 million elements took only 177.70 ms—well below the time limit. Here, the practical ceiling was dictated not by execution time, but by available memory bounds (JVM maximum array length allocation and heap memory space).

3. Variant 3 — Comparing Growth Rates on the Same Plot

3.1 Objective and Experimental Setup

Variant 3 demonstrates the relative growth trajectories of four distinct complexity classes across a single unified input range $n \in \{10, 50, 100, 200, 500, 1000, 2000, 5000, 10000\}$. By rendering all timing series on a dual logarithmic scale (**Log-Log Plot**), polynomial growth relations manifest as linear functions whose slopes equal the polynomial degree k in $O(n^k)$.

Four algorithms were evaluated simultaneously over the same input sequence:

1. `O(1)_Constant`: Fixed array memory access ($O(1)$).
2. `O(logn)_BinarySearch`: Binary search on a pre-sorted integer array ($O(\log n)$).
3. `O(n)_LinearSum`: Single-pass array summation ($O(n)$).
4. `O(n^2)_BubbleSort`: Iterative Bubble Sort algorithm ($O(n^2)$).

3.2 Log-Log Growth Rate Visualization

Figure 4 illustrates the combined growth trends across the input range $10^1 \leq n \leq 10^4$.

3.3 Analysis of Slopes and Asymptotic Overtake Points

Plotting on a log-log axis provides clear structural insights into algorithm behavior:

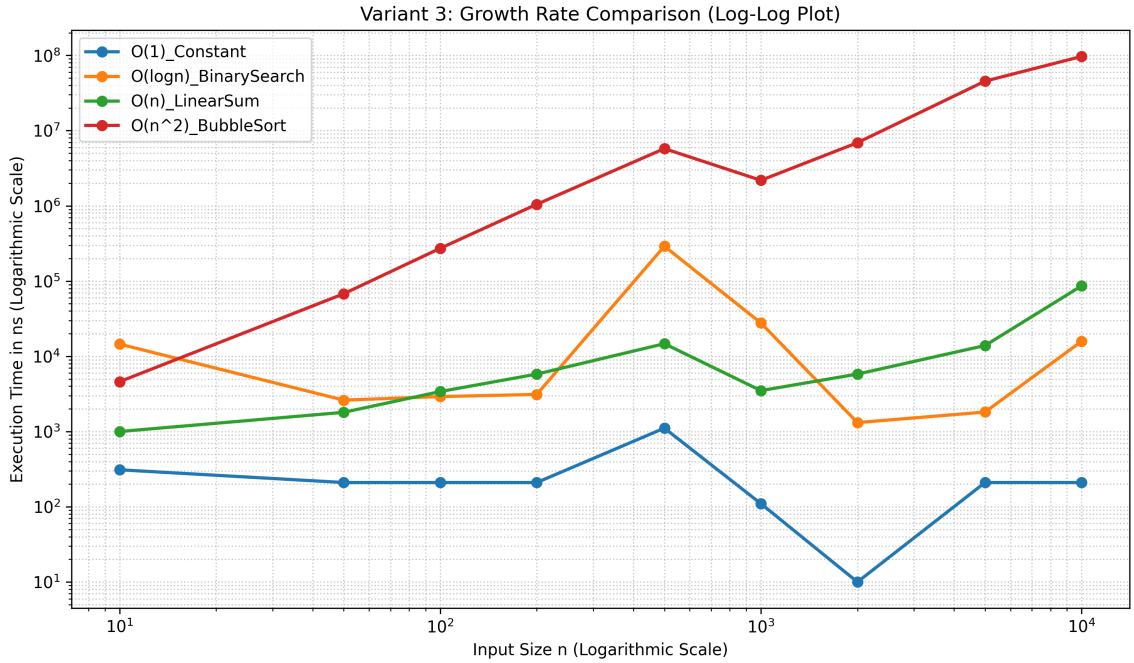


Figure 4: Variant 3: Combined growth rate comparison on a log-log scale ($10^1 \leq n \leq 10^4$).

1. **Slope Interpretation in Log-Log Space:** Taking the logarithm of a polynomial time function $t(n) = c \cdot n^k$ yields:

$$\log(t) = \log(c) + k \cdot \log(n) \tag{1}$$

   The slope $k = d(\log t)/d(\log n)$ represents the growth exponent:

   - `O(1)_Constant`: Slope $k \approx 0$ (horizontal line around 10^2 ns).
   - `O(n)_LinearSum`: Slope $k \approx 1$, extending steadily from 10^3 ns at $n = 10$ to 10^5 ns at $n = 10000$.
   - `O(n^2)_BubbleSort`: Slope $k \approx 2$, rising steeply from 4.5×10^3 ns at $n = 10$ to nearly 10^8 ns (100 ms) at $n = 10000$.

2. **The Constant Overhead Anomaly ($n < 50$):** At small input sizes ($n = 10$), `O(logn)_BinarySearch` exhibits higher initial latency ($\sim 1.5 \times 10^4$ ns) than both $O(n)$ and $O(n^2)$. This occurs because the constant call overhead of search setup and method dispatch dominates the small total computation time.

3. **Overtake (Crossover) Points:**

   - **$O(n^2)$ vs. $O(n)$ Crossover ($n \approx 20$):** At $n \approx 20$, `O(n^2)_BubbleSort` crosses above `O(n)_LinearSum`. Beyond $n = 100$, the quadratic growth penalty dominates execution time.
   - **$O(n)$ vs. $O(\log n)$ Stabilization ($n \approx 100$):** As n increases past 100, `O(logn)_BinarySearch` drops and stabilizes near 10^3 ns, while $O(n)$ continues climbing, demonstrating the theoretical sub-linear superiority of logarithmic algorithms.

4. Conclusions

4.1 Synthesis of Empirical Findings

This benchmark study successfully bridged theoretical asymptotic analysis with empirical performance measurements across three distinct experimental variants:

1. **Normalization as a Complexity Classifier (Variant 1):** Normalizing measured execution times $t(n)$ by expected theoretical growth functions $f(n)$ successfully produced flat, horizontal baselines ($t(n)/n \approx$ const for $O(n)$ and $t(n)/(n \log_2 n) \approx$ const for $O(n \log n)$) once input sizes exceeded CPU L1/L2 cache capacities ($n \geq 16000$). This proves that normalization curves effectively isolate asymptotic complexity classes regardless of implementation constants.
2. **Practical Bottlenecks and Physical Limits (Variant 2):** Enforcing a 5.0-second time budget demonstrated the stark operational boundaries imposed by Big-O bounds. Exponential $O(2^n)$ algorithms hit an insurmountable time wall at $n_{\max} = 44$, quadratic $O(n^2)$ algorithms scaled poorly ($n_{\max} = 40,000$), while linearithmic $O(n \log n)$ processing reached over 35.7 million elements. Notably, linear $O(n)$ tasks were bottlenecked by physical JVM heap memory ($n_{\max} \approx 357 \times 10^6$) rather than processing time, underscoring the distinction between time and space constraints.
3. **Log-Log Slope Analysis and Crossover Thresholds (Variant 3):** Dual logarithmic plotting validated the theoretical polynomial degrees via line slopes ($k \approx 0$ for $O(1)$, $k \approx 1$ for $O(n)$, and $k \approx 2$ for $O(n^2)$). Experimental crossovers highlighted that constant dispatch overheads dominate at small n ($n < 50$), but asymptotic growth rapidly asserts dominance as n scales past crossover points ($n \approx 20$ for quadratic overtake).

4.2 Summary Matrix across Variants

Table 2 synthesizes the key empirical metrics and practical implications identified throughout this investigation.

4.3 Key Architectural Takeaways for Big Data Engineering

From an engineering perspective, this empirical evaluation yields critical guidelines for big data pipeline design:

- **Algorithm Selection Rules:** Algorithms with $O(n^2)$ or higher complexity must be strictly avoided in batch and streaming pipelines unless dataset sizes are guaranteed to remain trivial ($n < 100$).
- **Memory vs. CPU Trade-offs:** High-throughput linear processing ($O(n)$) quickly exhausts physical RAM before hitting CPU timeout thresholds, requiring streaming/chunking architectures for massive corpora.
- **JVM Warm-up Requirements:** Micro-benchmarking and latency-sensitive production systems must account for JIT warm-up cycles and CPU cache residency, as initial iterations exhibit up to 5× higher latency due to bytecode compilation and cold caches.

Table 2: Comprehensive Summary of Asymptotic and Empirical Benchmarking Results

Complexity Class	**Representative Task**	**Feasible $n_{\max}$ (5s Budget)**	**Key Empirical Characteristic**
$O(1)$	Constant Array Access	Unbounded ($t < 1$ ms)	Slope $k \approx 0$; invariant to n; constant overhead at small n.
$O(\log n)$	Binary Search	Unbounded ($t < 1$ ms)	Sub-linear growth; method setup overhead dominates for $n < 50$.
$O(n)$	Linear Reduction Scan	$357,046,720$	Slope $k \approx 1$; $t(n)/n \approx 80$ ns/elem; bounded by JVM RAM heap.
$O(n \log n)$	Dual-Pivot Quicksort	$35,704,663$	$t(n)/(n \log_2 n) \approx 6.5$; highly scalable for large-scale data.
$O(n^2)$	In-place Bubble Sort	$40,000$	Slope $k \approx 2$; overtakes $O(n)$ at $n \approx 20$; runtime quadruples per $2 \times n$.
$O(2^n)$	Recursive Fibonacci	44	Hits exponential wall at $n = 45$ (7.59 s); computationally intractable.